Lecture 2

Gradient decent (GD) and its variants

- Books:
  - Optimization for Engineering Design: Kalyanmoy Deb
  - Algorithms for Optimization : Mykel J. Kochenderfer and Tim A. Wheeler
    - https://algorithmsbook.com/optimization/files/optimization.pdf
  - Engineering Design Optimization: Joaquim R. R. A. Martins, Andrew Ning
  - https://flowlab.groups.et.byu.net/mdobook.pdf

Gradient decent (GD)

In GD:

- Decent direction:

$$d^{(k)} = -\nabla f\left(x^{(k)}\right)$$

- Step size $\alpha^{(k)}$:
- $\alpha^{(k)}$ = constant for all k (Basic GD model)
- Position update equation:

$$x^{(k+1)} = x^{(k)} - \alpha^{(k)} \nabla f\left(x^{(k)}\right)$$

GD:

Gradient decent for function optimization

%Inputs

$f: \mathbb{R}^n \to \mathbb{R}$, a differentiable loss function

$x^{(0)}$: The initial guess for optimization solution

%Output: x^* a local minima of the loss function

Begin

$k \leftarrow 0$

While (stopping criteria like $\left\|\nabla f\left(x^{(k)}\right)\right\| > \varepsilon$**) do**

$\alpha^{(k)} = \alpha$ (constant)

$x^{(k+1)} = x^{(k)} - \alpha^{(k)} \nabla f\left(x^{(k)}\right)$

$k = k + 1;$

EndWhile

End

Gradient decent in 2 D space with constant α

%Inputs

$f: \mathbb{R}^2 \to \mathbb{R}$, a smooth loss function

$x^{(0)} = (x_1, x_2)$: The initial guess for optimization solution

%Output: $x^* = (x_1^*, x_2^*)$ a local minima of the loss function

Begin

$k \leftarrow 0$

While (stopping criteria $\left\|\nabla f\left(x_1^{(k)}, x_2^{(k)}\right)\right\| > \varepsilon$**) do**

- Compute $\nabla f\left(x_1^{(k)}, x_2^{(k)}\right) = \left[\frac{\partial f}{\partial x_1^{(k)}}, \frac{\partial f}{\partial x_2^{(k)}}\right]$
- $\alpha^{(k)} = \alpha$ (constant)
- Update the components

$$x_1^{(k+1)} = x_1^{(k)} - \alpha \times \frac{\partial f}{\partial x_1^{(k)}}$$

$$x_2^{(k+1)} = x_2^{(k)} - \alpha \times \frac{\partial f}{\partial x_2^{(k)}}$$

- Update the position

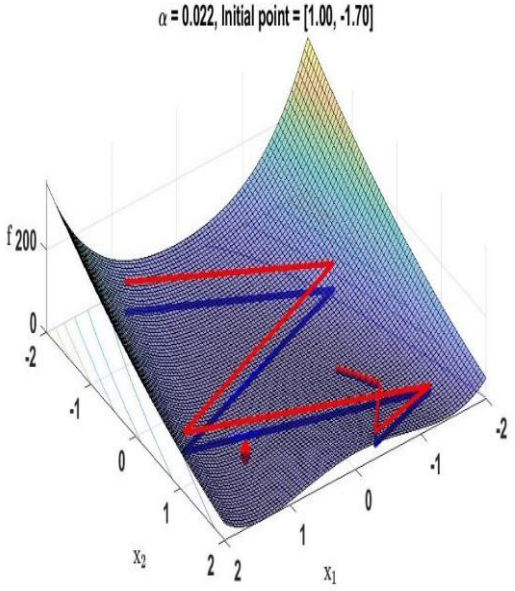
$x^{(k+1)} = \left[x_1^{(k+1)}, x_2^{(k+1)}\right]$

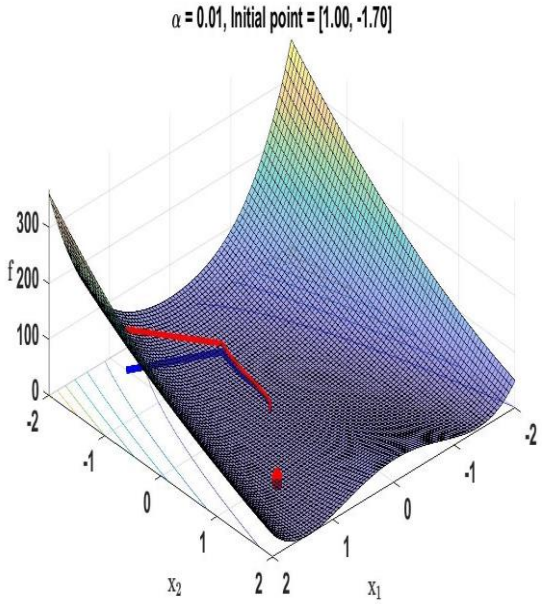
- $k = k + 1;$

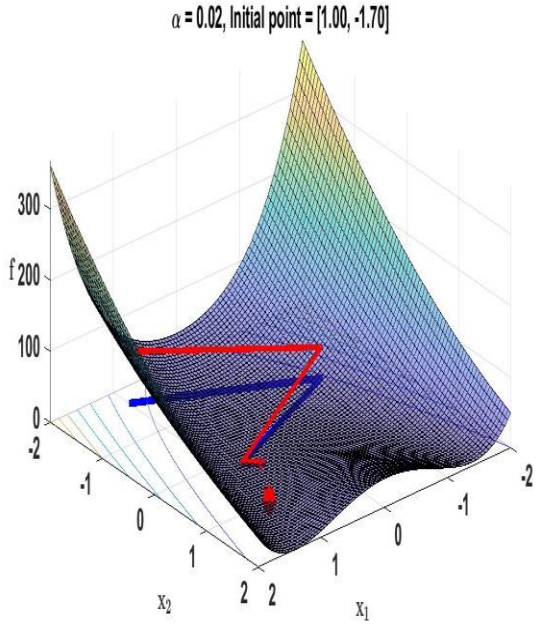
EndWhile

End

Gradient decent over Rosenbrock







- **Fixed α limitations:**
- If α is too small, convergence can be slow.
- If α is too large, the algorithm might overshoot the minimum, leading to divergence.

Line search methods for adaptive α

$$x^{(k+1)} \leftarrow x^{(k)} + \alpha^{(k)} d^{(k)}, \qquad k = 0,1,2,3, \ldots.$$

$\alpha^{(k)} = \alpha$
Fixed step size

For optimal $\alpha^{(k)}$

- **Inexact line search**
  - Wolfe conditions
    - Armijo rule
    - Backtracking line search

For optimal $\alpha^{(k)}$

- **Exact Line search**: as a **univariate optimization problem**:
- $\phi(\alpha) = f(x^{(k)} + \alpha^{(k)} d^{(k)})$
- $\alpha^{(k)} = \arg\min_{\alpha>0} \phi(\alpha) = \arg\min_{\alpha>0} f(x^{(k)} + \alpha^{(k)} d^{(k)})$

Exact Line search for optimal α:

$$\alpha^{(k)} = \arg\min_{\alpha>0} \phi(\alpha) = f(x^{(k)} + \alpha(-\nabla f(x^{(k)})),$$

$$x^{(k+1)} \leftarrow x^{(k)} + \alpha^{(k)} \left(-\nabla f\left(x^{(k)}\right)\right), k = 0,1,2,3, \ldots.$$

Limitation:
Computational cost: evaluating the objective function at multiple points along the direction of the negative gradient, which can be time-consuming.

Gradient decent in 2 D space with exact line search

%Inputs
$f: \mathbb{R}^2 \rightarrow \mathbb{R}$, a smooth loss function
$x^{(0)} = (x_1, x_2)$: The initial guess for optimization solution
%Output: $x^* = (x_1^*, x_2^*)$ a local minima of the loss function
Begin
$k \leftarrow 0$
While (stopping criteria $\left\|\nabla f\left(x_1^{(k)}, x_2^{(k)}\right)\right\| > \varepsilon$) **do**

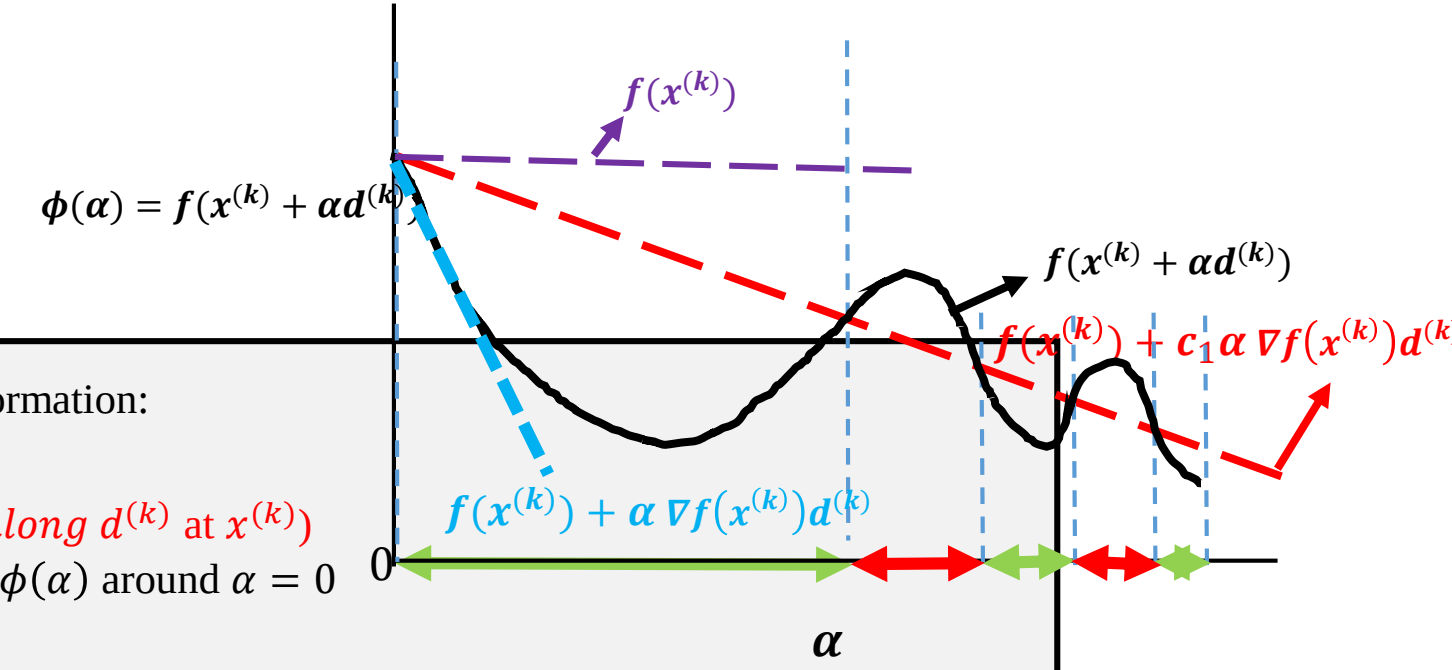
- Compute $\nabla f\left(x_1^{(k)}, x_2^{(k)}\right) = \left[\frac{\partial f}{\partial x_1^{(k)}}, \frac{\partial f}{\partial x_2^{(k)}}\right]$
- $\alpha^{(k)} = \arg\min_{\alpha \in \mathbb{R}_+} f\left((x_1^{(k)}, x_2^{(k)}) - \alpha \nabla f\left(x_1^{(k)}, x_2^{(k)}\right)\right)$
- Update the components
  $x_1^{(k+1)} = x_1^{(k)} - \alpha^{(k)} \times \frac{\partial f}{\partial x_1^{(k)}}$
  $x_2^{(k+1)} = x_2^{(k)} - \alpha^{(k)} \times \frac{\partial f}{\partial x_2^{(k)}}$
- Update the position
  $x^{(k+1)} = \left[x_1^{(k+1)}, x_2^{(k+1)}\right]$
- $k = k + 1$;

EndWhile
End

Armijo (sufficient decrease) condition

(First Wolfe condition)

- $\phi(\alpha) = f\left(x^{(k)} + \alpha^{(k)} d^{(k)}\right)$
- $\phi'(\alpha) = \frac{d}{d\alpha}\phi(\alpha) = \nabla f\left(x^{(k)} + \alpha^{(k)} d^{(k)}\right)^T d^{(k)}$



- At a current position $x^{(k)}$, where $\alpha = 0$, we have two information:
  $\phi(0) = f\left(x^{(k)}\right)$
  $\phi'(0) = \nabla f\left(x^{(k)}\right)^T d^{(k)}$ (Directional derivative of f *along* $d^{(k)}$ at $x^{(k)}$)
- This information helps to form a linear approximation of $\phi(\alpha)$ around $\alpha = 0$
  - $\phi(\alpha) \approx \phi(0) + \alpha\, \phi'(0)$
  - $f\left(x^{(k)} + \alpha^{(k)} d^{(k)}\right) \approx f(x^{(k)}) + \alpha \nabla f\left(x^{(k)}\right)^T d^{(k)}$
- Since, at the current point $x^{(k)}$, $\nabla f\left(x^{(k)}\right)^T d^{(k)} < 0$ (it means decent direction). Therefore for a small positive α, $\alpha \nabla f\left(x^{(k)}\right)^T d^{(k)}$ is negative
- It means $f\left(x^{(k)} + \alpha^{(k)} d^{(k)}\right) < f(x^{(k)})$
- The predicted decrease is: $\Delta_{pred} = -\alpha \nabla f\left(x^{(k)}\right)^T d^{(k)} = \alpha \left|\nabla f\left(x^{(k)}\right)^T d^{(k)}\right|$
- The actual decrease is: $\Delta_{Actual} = f\left(x^{(k)}\right) - f\left(x^{(k)} + \alpha^{(k)} d^{(k)}\right)$
- Is the actual decrease sufficient?
  - $\Delta_{Actual} \geq \Delta_{pred}$ (too strict because the linear model is only an approximation)
  - The actual decrease should be at least some fraction of the predicted decrease. It means $\Delta_{Actual} \geq \mathbf{c_1} \Delta_{pred}$

Armijo (sufficient decrease) condition

$$\Delta_{Actual} \geq c_1 \Delta_{pred}$$

$$f(x^{(k)} + \alpha d^{(k)}) - f(x^{(k)}) \geq c_1 \alpha |\nabla f(x^{(k)})^T d^{(k)}|, c_1 \in (0,1)$$

$$-f(x^{(k)} + \alpha d^{(k)}) + f(x^{(k)}) \leq -c_1 \alpha |\nabla f(x^{(k)})^T d^{(k)}|, c_1 \in (0,1)$$

The condition for sufficient decrease, also called the Armijo rule, states:

$$f(x^{(k)} + \alpha d^{(k)}) \leq f(x^{(k)}) + c_1 \alpha \nabla f(x^{(k)})^T d^{(k)}, c_1 \in (0,1)$$

- $\left|c_1 \nabla f(x^{(k)})^T d^{(k)}\right| < \left|\nabla f(x^{(k)})^T d^{(k)}\right|$
- *c_1 "flattens" the line because it scales the tangent slope to a fraction of the actual decrease, making the required descent less steep.*

Limitation:
Very small values of $\alpha^{(k)}$ can satisfy this condition

Backtracking line search

Backtracking line search

%Inputs
Given α_{init} (for example $\alpha_{\text{init}} = 1$)
Given $\beta = (0,1)$ (for example $\beta = 0.4$)
Given k^{th} iterate
%Output: Optimal step size $(\boldsymbol{\alpha^{(k)}})$
Begin
$l \leftarrow 0$
$\boldsymbol{\alpha^{(0)}} \leftarrow \alpha_{\text{init}}$
While $(f(x^{(k)} + \alpha^{(l)} d^{(k)}) > f(x^{(k)}))$ **do**
- Set $\alpha^{(l+1)} = \beta \alpha^{(l)}$
- $l \leftarrow l + 1$

EndWhile
- $\boldsymbol{\alpha^{(k)}} \leftarrow \boldsymbol{\alpha^{(l)}}$

End

Backtracking with Armijo rule

%Inputs
Given α_{init} (for example $\alpha_{\text{init}} = 1$)
Given $c_1 = \beta = (0,1)$ (for example $\beta = 0.4, c_1 = 10^{-4}$)
Given k^{th} iterate
%Output: Optimal step size $(\boldsymbol{\alpha^{(k)}})$
Begin
$l \leftarrow 0$
$\boldsymbol{\alpha^{(0)}} \leftarrow \alpha_{\text{init}}$
While $(f(x^{(k)} + \alpha^{(l)} d^{(k)}) > f(x^{(k)}) + c_1 \alpha^{(l)} \nabla f(x^{(k)})^T d^{(k)})$ **do**
- Set $\alpha^{(l+1)} = \beta \alpha^{(l)}$
- $l \leftarrow l + 1$

EndWhile
- $\boldsymbol{\alpha^{(k)}} \leftarrow \boldsymbol{\alpha^{(l)}}$

End

Armijo rule:
$f(x^{(k)} + \alpha d^{(k)}) \leq f(x^{(k)}) + c_1 \alpha \nabla f(x^{(k)})^T d^{(k)}, c_1 \in (0,1)$

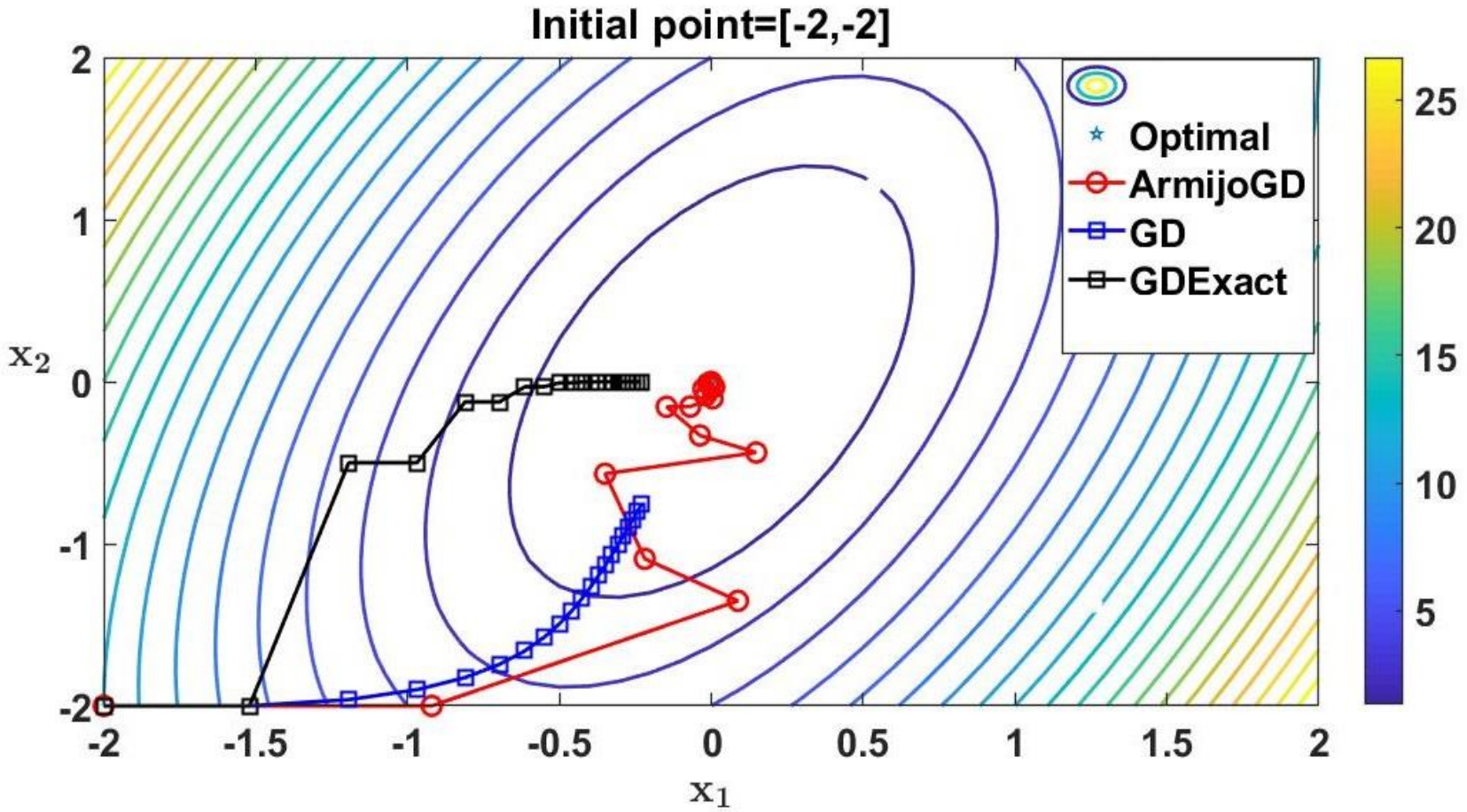
GD with backtracking Armijo rule

GD with Armijo backtracking

%Inputs
$f: \mathbb{R}^n \to \mathbb{R}$, a smooth loss function
$x^{(0)}$: The initial guess for optimization solution
Initialize c_1, β both in $(0,1)$
%Output: x^* a local minima of the loss function
Begin
$k \leftarrow 0$
While (stopping criteria like $\|\nabla f(x^{(k)})\| > \varepsilon$) **do**
$\alpha^{(k)} = 1$: Initial learning rate
While $\left(f\left(x^{(k)} - \alpha \nabla f(x^{(k)})\right) > f(x^{(k)}) - c_1 \alpha^{(k)} \nabla f(x^{(k)})^T \nabla f(x^{(k)})\right)$
$\alpha^{(k)} = \alpha^{(k)} * \beta$
EndWhile
$x^{(k+1)} = x^{(k)} - \alpha^{(k)} \nabla f(x^{(k)})$
$k = k + 1;$
EndWhile
End

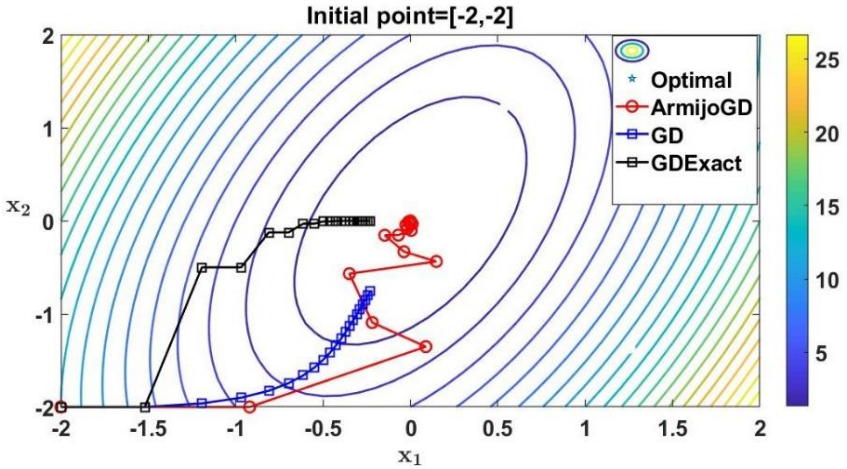
GD with Armijo rule

1



$$f(x_1, x_2) = 4x_1^2 + x_2^2 - 2x_1x_2$$

Flaws in GD



$$f(x_1, x_2) = 4x_1^2 + x_2^2 - 2x_1x_2$$

- A key issue with GD is "**zig-zagging**".
- In a long, narrow, shallow ravine with steep sides, GD tends to oscillate across the **ravine** rather than progressing steadily towards the optimum.
- The negative gradient points primarily across the ravine's narrow width, causing the algorithm to zigzag between the steep walls rather than making efficient progress towards the minimum. This results in **slow convergence**.

zig-zagging

- 1

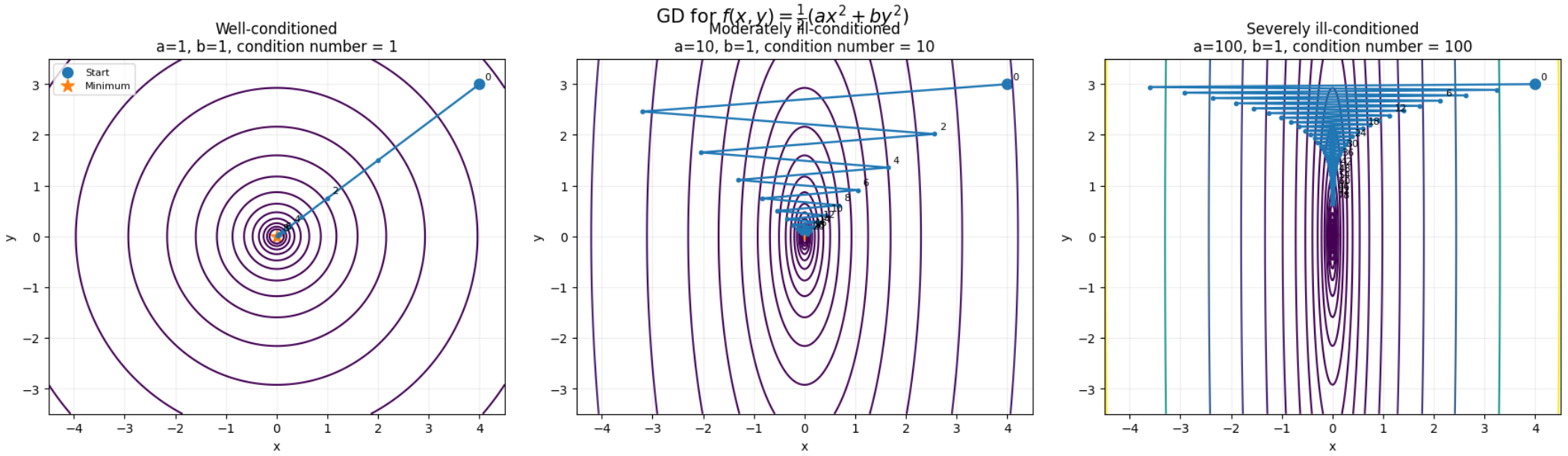
- A key issue with Gradient Descent (GD) is **zig-zagging.**
- Zig-zagging is prominent when the objective function is **ill-conditioned,** producing a long, narrow ravine.
- An ill-conditioned problem has very different curvature in different directions:
  - High curvature across the ravine → steep walls
  - Low curvature along the ravine → flat direction
- The negative gradient points primarily across the ravine, because that is the direction of steepest descent.
- Consequently, GD repeatedly oscillates between the steep walls instead of progressing efficiently along the ravine toward the optimum.
- This side-to-side movement results in slow convergence.

- Let $f(x,y) = \frac{1}{2}(ax^2 + by^2)$
- Calculate $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]^T = [\text{ax}, \text{by}]$
- Calculate $H = \nabla^2 f = \begin{bmatrix} \frac{\partial^2 f}{\partial x \partial x} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y \partial y} \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & b \end{bmatrix}$
- Condition number $c = \frac{Largest\ eigenvlaue\ of\ H}{smallest\ eigenvlaue\ of\ H}$

- Case 1: When $a = 1. b = 1$
- $\Rightarrow H = \nabla^2 f = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
- Eigenvalues: $\lambda_1 = \lambda_2 = 1$
- Condition number $c = \frac{1}{1} = 1$
- The curvature is equal in both directions. (well conditioned)

- Case 2: When $a = 10. b = 1$
- $\Rightarrow H = \nabla^2 f = \begin{bmatrix} 10 & 0 \\ 0 & 1 \end{bmatrix}$
- Eigenvalues: $\lambda_1 = 10, \lambda_2 = 1$
- Condition number $c = \frac{10}{1} = 10$
- The curvature in the x-direction is 10 times larger than in the y-direction. (ill conditioned)

- Case 3: When $a = 100. b = 1$
- $\Rightarrow H = \nabla^2 f = \begin{bmatrix} 100 & 0 \\ 0 & 1 \end{bmatrix}$
- Eigenvalues: $\lambda_1 = 100, \lambda_2 = 1$
- Condition number $c = \frac{100}{1} = 100$ (Severely ill-conditioned)
- The curvature in the x-direction is 100 times larger than in the y-direction.



$$x^{(0)} = (4,3), \alpha = 0.5(case\ 1), 0.18\ (case\ 2), 0.019\ (case\ 3)$$

Momentum

To accelerate gradient descent by incorporating past gradients, which helps to smooth out the trajectory and potentially speed up convergence.

- **GD is memoryless mechanism**
- **Momentum GD: Memory based GD**
- The current update in position, also look at the history of updates.
- The momentum GD update equations are:

$$v^{(k)} = \gamma v^{(k-1)} + \alpha \nabla f\left(x^{(k)}\right)$$
$$x^{(k+1)} = x^{(k)} - v^{(k)}, \qquad k = 1,2, \ldots\ldots$$

  - $v^{(k)}$ is the velocity (or momentum) at k^{th} iteration
  - $\gamma \in (0,1)$ is a momentum coefficient
  - α is a learning rate
  - $\nabla f\left(x^{(k)}\right)$: Function gradient at $x^{(k)}$ position
  - If $\gamma = 0$, GD is recovered.

Momentum

The momentum GD update equations are:

$$v^{(k)} = \gamma v^{(k-1)} + \alpha \nabla f\left(x^{(k)}\right)$$
$$x^{(k+1)} = x^{(k)} - v^{(k)}, \qquad k = 1,2, \ldots\ldots$$

- Initial velocity (momentum) $v^{(0)} = 0$,
- $v^{(1)} = \gamma v^{(0)} + \alpha \nabla f\left(x^{(1)}\right) = \alpha \nabla f\left(x^{(1)}\right)$
- $v^{(2)} = \gamma v^{(1)} + \alpha \nabla f\left(x^{(2)}\right)$
  $= \gamma \left(\alpha \nabla f\left(x^{(1)}\right)\right) + \alpha \nabla f\left(x^{(2)}\right)$
- $v^{(3)} = \gamma v^{(2)} + \alpha \nabla f\left(x^{(3)}\right)$
  $= \gamma \left(\gamma \left(\alpha \nabla f\left(x^{(1)}\right)\right) + \alpha \nabla f\left(x^{(2)}\right)\right) + \alpha \nabla f\left(x^{(3)}\right)$
  $= \gamma^2 \alpha \nabla f\left(x^{(1)}\right) + \gamma \alpha \nabla f\left(x^{(2)}\right) + \alpha \nabla f\left(x^{(3)}\right)$
- $v^{(4)} = \gamma^3 \alpha \nabla f\left(x^{(1)}\right) + \gamma^2 \alpha \nabla f\left(x^{(2)}\right) + \gamma \alpha \nabla f\left(x^{(3)}\right) + \alpha \nabla f\left(x^{(4)}\right)$

Similarly,

- $v^{(k)} = \gamma^{k-1} \alpha \nabla f\left(x^{(1)}\right) + \gamma^{k-2} \alpha \nabla f\left(x^{(2)}\right) + \cdots \gamma \alpha \nabla f\left(x^{(k-1)}\right) + \cdots + \alpha \nabla f\left(x^{(k)}\right)$

Finally,

- $x^{(k+1)} = x^{(k)} - \left(\gamma^{k-1} \alpha \nabla f\left(x^{(1)}\right) + \gamma^{k-2} \alpha \nabla f\left(x^{(2)}\right) + \cdots \gamma \alpha \nabla f\left(x^{(k-1)}\right) + \cdots + \alpha \nabla f\left(x^{(k)}\right)\right)$ % **γ exponentially decay rate**

Momentum

Momentum based GD for function optimization

%Inputs

$f: \mathbb{R}^n \rightarrow \mathbb{R}$, a smooth loss function

$x^{(0)} \in \mathbb{R}^n$ The initial guess for optimization solution

$v^{(0)} = 0_n \in \mathbb{R}^n$: Initial Momentum (zero n dimensional vector)

$\gamma \in (0,1)$: Momentum coefficient

α: Learning rate

%Output: $x^* \in \mathbb{R}^n$ a local minima of the loss function f

Begin

$k \leftarrow 1$

While (stopping criteria $\left\|\nabla f\left(x^{(k)}\right)\right\| > \varepsilon$) **do**

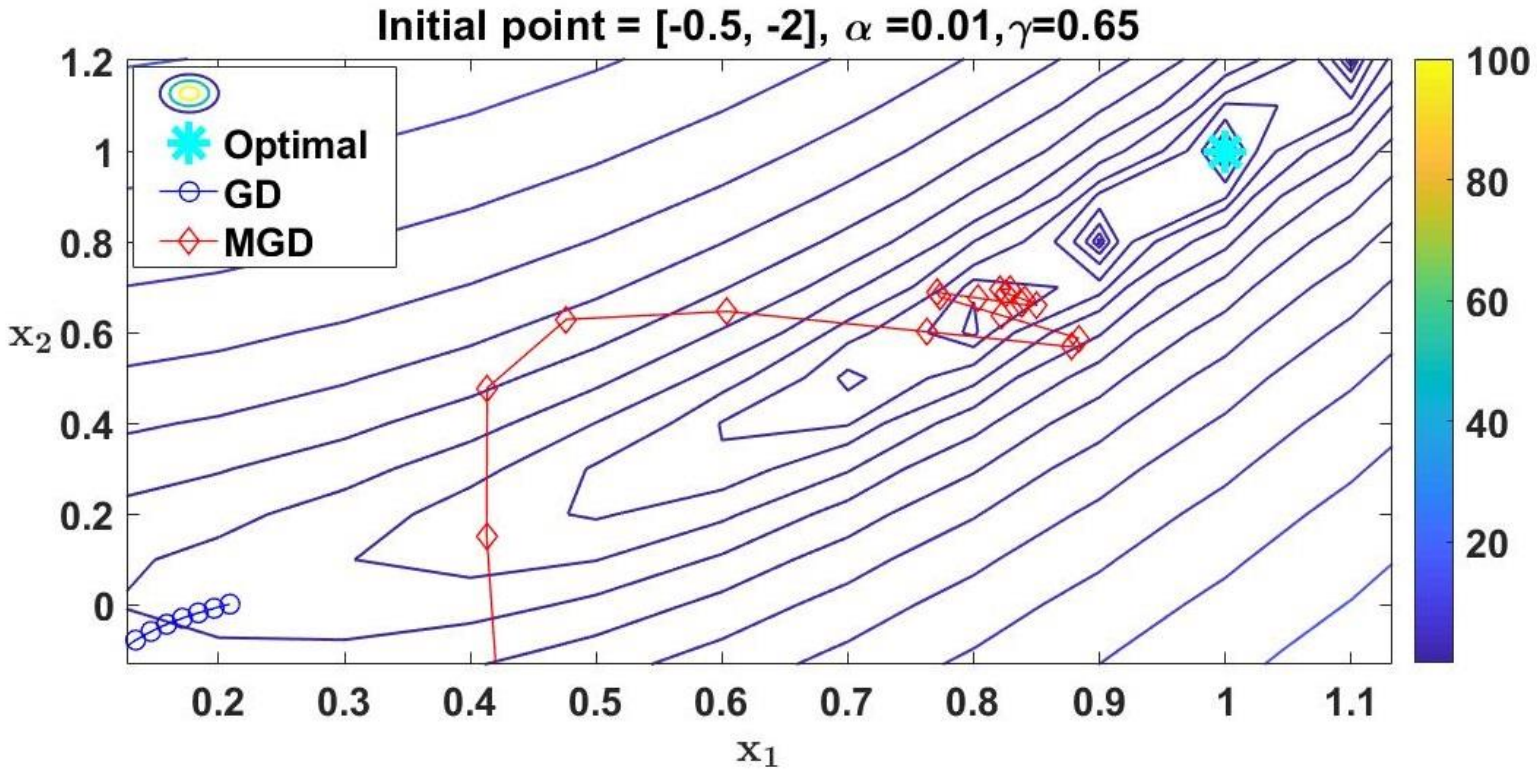
- Compute $\nabla f\left(x^{(k)}\right)$
- $v^{(k)} = \gamma * v^{(k-1)} + \alpha \nabla f\left(x^{(k)}\right)$ % Accumulate gradients
- $x^{(k+1)} = x^{(k)} - v^{(k)}$ %Update position
- $k = k + 1;$

EndWhile

End

Shortcomings of Momentum GD:

- Oscillations
- Two hyper-parameters
- **Overshoot issue**



Nesterov accelerated gradient descent (NGD)

- The position update equations of Momentum based GD:

$$v^{(k)} = \gamma v^{(k-1)} + \alpha \nabla f\left(x^{(k)}\right)$$
$$x^{(k+1)} = x^{(k)} - v^{(k)}, \quad k = 1,2, \ldots..$$

- Or

$$x^{(k+1)} = x^{(k)} - \left(\gamma v^{(k-1)} + \alpha \nabla f\left(x^{(k)}\right)\right), \quad k = 1,2, \ldots..$$
$$x^{(k+1)} = \left(x^{(k)} - \gamma v^{(k-1)}\right) - \alpha \nabla f\left(x^{(k)}\right), \quad k = 1,2, \ldots..$$

$$x^{(k+1)} = \left(x^{(k)} - \gamma v^{(k-1)}\right) - \alpha \nabla f\left(x^{(k)} - \gamma v^{(k-1)}\right), \quad k = 1,2, \ldots$$

Position update equations of NGD:

$$v^{(k)} = \gamma v^{(k-1)} + \alpha \nabla f\left(x^{(k)} - \gamma v^{(k-1)}\right)$$
$$x^{(k+1)} = x^{(k)} - v^{(k)}, \quad k = 1,2, \ldots..$$

Nesterov accelerated gradient descent (NGD)

NGD for function optimization

%Inputs

$f: \mathbb{R}^n \to \mathbb{R}$, a smooth loss function

$x^{(0)} \in \mathbb{R}^n$ The initial guess for optimization solution

$v^{(0)} = 0_n \in \mathbb{R}^n$: Initial Momentum (zero n dimensional vector)

$\gamma \in (0,1)$: Momentum coefficient

α: Learning rate

%Output: $x^* \in \mathbb{R}^n$ a local minima of the loss function f

Begin

$k \leftarrow 1$

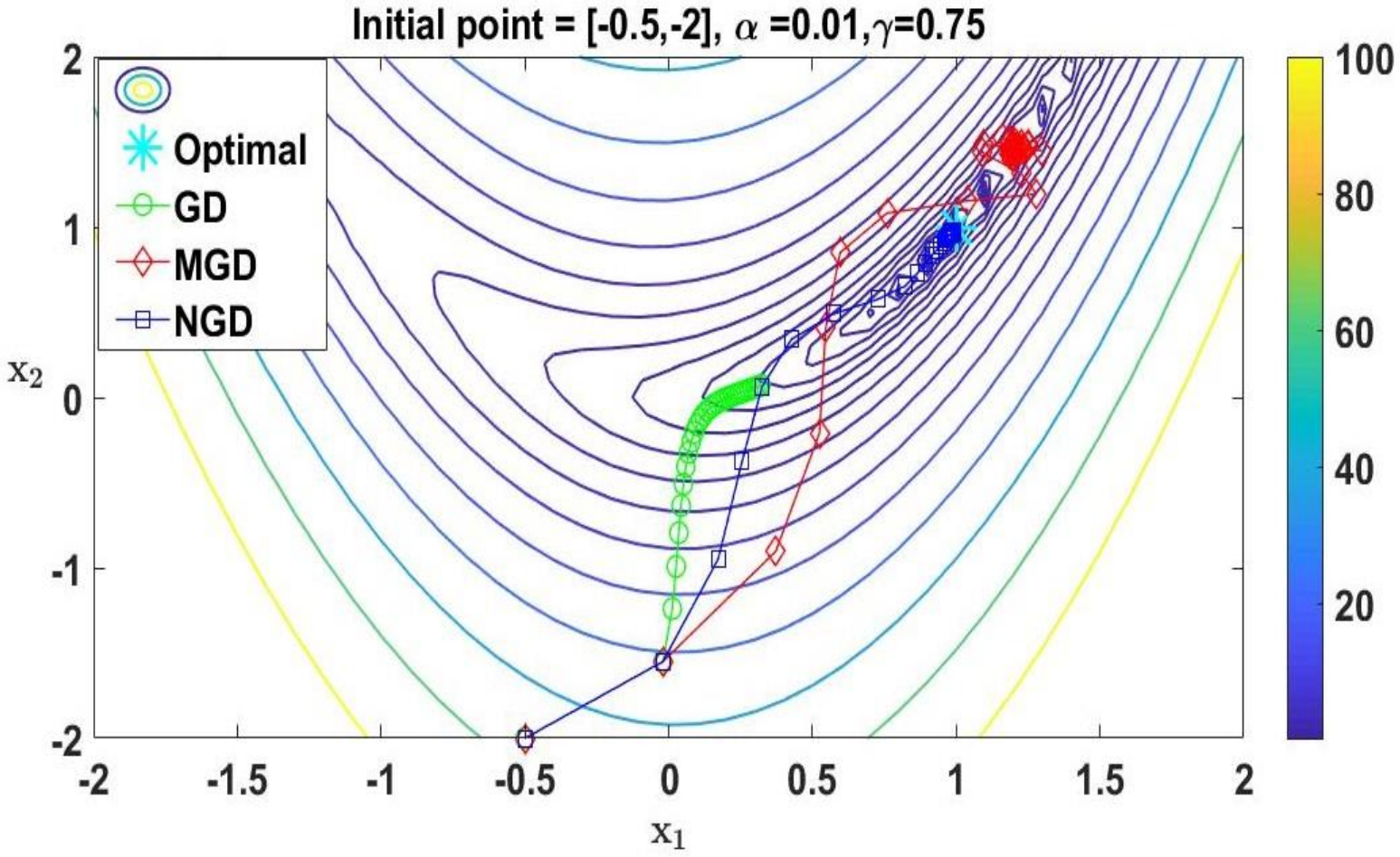
While (stopping criteria $\|\nabla f(x^{(k)})\| > \varepsilon$) **do**

- $x_{Lookhead} = (x^{(k)} - \gamma v^{(k-1)})$ %Compute look ahead position
- Compute $\nabla f(x_{Lookhead}) = \nabla f(x^{(k)} - \gamma v^{(k-1)})$ % Compute gradient at look ahead position
- $v^{(k)} = \gamma * v^{(k-1)} + \alpha \nabla f(x_{Lookhead})$ % Accumulate gradients
- $x^{(k+1)} = x^{(k)} - v^{(k)}$ %Update position
- $k = k + 1;$

EndWhile

End

NGD


Initial point = [-0.5,-2], α =0.01,γ=0.75
Optimal
GD
MGD
NGD
x2
x1
2
1
0
-1
-2
-2
-1.5
-1
-0.5
0
0.5
1
1.5
2
100
80
60
40
20